UNIVERSIDAD NACIONAL DE COSTA RICA

SEDE REGIONAL CHOROTEGA CAMPUS NICOYA

Curso: Programación 1 – EIF201

Proyecto: Implementación de Juego Blackjack por Consola

Estudiantes:

Sasha Castillo Montiel - 208770716

Daniela Venegas Zuñiga – 208440873

Krisaneth Lopez Gomez - 504600973

Profesor: Francisco Coulon Olliver

Fecha de Entrega: 13 de noviembre de 2026

1. Introducción

El desarrollo de software orientado a objetos constituye una de las metodologías fundamentales en la ingeniería de software actual, permitiendo modelar sistemas del mundo real mediante la abstracción, el encapsulamiento y modularidad. El presente documento detalla la primera fase del diseño y desarrollo de un sistema de simulación del juego de cartas Blackjack por consola, implementado en el lenguaje de programación C++.

A lo largo de este informe se establece el diseño estructural del sistema mediante diagramas de clases UML, la definición formal de las reglas del negocio, los casos de prueba prácticos y el avance de código correspondiente a las estructuras base del programa. Este proyecto no solo busca replicar las reglas lógicas y de apuestas de un casino tradicional, sino también aplicar buenas prácticas de control de versiones y trabajo colaborativo en equipo.

2. Definición del problema

El problema consiste en diseñar e implementar un sistema de software que se ejecute por consola y permita simular una partida de Blackjack mediante el lenguaje de programación C++. El sistema debe aplicar las reglas del juego, administrar las apuestas y controlar los turnos de los jugadores y del crupier.

Entre los principales retos que debe resolver el programa se encuentran:

Gestión de múltiples mazos: administrar un zapato compuesto por 6 mazos estándar de 52 cartas, para un total de 312 cartas. Los mazos deben barajarse una única vez al inicio de la partida y el sistema debe controlar la cantidad de cartas disponibles.

Cálculo de las puntuaciones: determinar correctamente el valor de cada mano, considerando que el As puede valer 1 u 11 puntos, según convenga para evitar superar los 21 puntos.

Administración de apuestas y fichas: controlar el saldo de cada jugador, validar las apuestas mínimas y máximas y comprobar que tenga fichas suficientes para doblar o dividir una mano.

Control de turnos: permitir que los jugadores realicen las acciones disponibles y automatizar el comportamiento del crupier, quien debe pedir cartas hasta alcanzar un mínimo de 17 puntos.

Resolución de las rondas: determinar victorias, derrotas y empates, aplicar los pagos correspondientes al Blackjack, al seguro y a las demás apuestas, y realizar los redondeos establecidos en las reglas del proyecto.

El sistema también debe validar las entradas del usuario y controlar situaciones especiales, como quedarse sin cartas en el zapato, pasarse de 21 puntos o no contar con saldo suficiente para realizar una acción.

3. Reglas del juego y ejemplos de uso

3.1. Reglas generales

El Blackjack se desarrolla de acuerdo con las siguientes reglas:

Objetivo: obtener una puntuación superior a la del crupier sin superar los 21 puntos.

Configuración: se utilizan 6 mazos estándar de 52 cartas, para un total de 312 cartas. Se barajan una sola vez al inicio de la partida y no se vuelven a barajar.

Fichas y apuestas: cada jugador comienza con 1000 fichas. La apuesta mínima es de 10 fichas y la máxima es de 500 fichas. Estos valores deben poder modificarse mediante constantes.

Valor de las cartas: las cartas del 2 al 10 conservan su valor numérico; las figuras J, Q y K valen 10 puntos; el As puede valer 1 u 11 puntos, según convenga para no superar los 21.

Reparto inicial: cada jugador recibe dos cartas boca arriba. El crupier recibe una carta visible y otra boca abajo.

Blackjack natural: ocurre cuando las dos primeras cartas de una mano suman exactamente 21 puntos. Si el jugador obtiene Blackjack y el crupier no, recibe un pago de 3:2. Si ambos obtienen Blackjack, se produce un empate y se devuelve la apuesta.

Revisión del Blackjack del crupier: si la carta visible del crupier es un As o tiene un valor de 10, debe comprobarse si la carta oculta completa un Blackjack. Si lo tiene, la ronda termina de acuerdo con las reglas de pago correspondientes.

Acciones del jugador: durante su turno puede pedir una carta, plantarse, doblar o dividir, siempre que cumpla las condiciones y tenga saldo suficiente.

Doblar: solo está permitido después de recibir las dos cartas iniciales. El jugador duplica su apuesta, recibe una única carta adicional y debe plantarse.

Dividir: se permite cuando las dos cartas iniciales tienen el mismo valor. El jugador realiza una segunda apuesta igual a la original y juega cada mano de manera independiente. Cada mano recibe una carta adicional.

Seguro: solamente puede solicitarse cuando el crupier muestra un As. La apuesta de seguro puede ser de hasta la mitad de la apuesta original. Si el crupier tiene Blackjack, el seguro paga 2:1; de lo contrario, se pierde esa apuesta adicional.

Turno del crupier: después de que los jugadores finalizan sus acciones, el crupier revela su carta oculta y pide cartas hasta alcanzar 17 puntos o más.

Resolución: si el jugador supera los 21 puntos, pierde su apuesta. Si el crupier se pasa de 21 y el jugador no, gana el jugador. Si ninguno se pasa, se comparan las puntuaciones: la mayor gana, la menor pierde y las iguales producen un empate, excepto en los casos de Blackjack natural, que tienen su propia regla.

Pagos: las victorias normales pagan 1:1; el Blackjack natural paga 3:2; el seguro paga 2:1; los empates devuelven la apuesta. Los pagos decimales se redondean hacia abajo.

Agotamiento del zapato: si se terminan las cartas disponibles, la partida finaliza al concluir la ronda que se encuentra en curso.

3.2. Ejemplos de uso

Ejemplo 1: Victoria estándar (pago 1:1)

Cartas iniciales del jugador: [9, 8]. Total: 17 puntos. El jugador decide plantarse.

Cartas del crupier: [10, 6]. Total inicial: 16 puntos. El crupier debe pedir una carta y recibe un 8, alcanzando un total de 24 puntos.

Resultado: el crupier supera los 21 puntos y el jugador no. Por lo tanto, el jugador gana la ronda.

Apuesta y pago: apuesta inicial de ₡100. El jugador obtiene ₡100 de ganancia neta y recupera su apuesta original. Recibe un total de ₡200.

Ejemplo 2: Derrota por superar los 21 puntos (Bust)

Cartas iniciales del jugador: [10, 5]. Total: 15 puntos. El jugador decide pedir una carta adicional y recibe una J, cuyo valor es 10.

Total de la mano: 10 + 5 + 10 = 25 puntos.

Resultado: el jugador supera los 21 puntos y pierde la ronda inmediatamente, independientemente del resultado posterior del crupier.

Apuesta y pago: apuesta inicial de ₡50. El jugador pierde la apuesta y no recibe devolución.

Ejemplo 3: Empate (Push)

Cartas finales del jugador: [10, 8]. Total: 18 puntos. El jugador decide plantarse.

Cartas finales del crupier: [9, 9]. Total: 18 puntos. El crupier se planta al alcanzar 17 puntos o más.

Resultado: ambas manos tienen la misma puntuación y no se trata de un Blackjack natural. Se declara un empate.

Apuesta y pago: apuesta inicial de ₡200. Se devuelve la totalidad de la apuesta al jugador, sin ganancias ni pérdidas.

Ejemplo 4: Blackjack natural (pago 3:2)

Cartas iniciales del jugador: [As, K]. Total: 21 puntos con las dos primeras cartas.

Cartas iniciales del crupier: [6, 8]. Total: 14 puntos. El crupier no tiene Blackjack y continúa jugando hasta alcanzar 17 puntos o más.

Resultado: el jugador obtiene un Blackjack natural y el crupier no.

Apuesta y pago: apuesta inicial de ₡100. El Blackjack paga 3:2, por lo que el jugador obtiene ₡150 de ganancia y recupera su apuesta original. Recibe un total de ₡250.

Ejemplo 5: División de manos (Split)

Cartas iniciales del jugador: [8, 8]. Total: 16 puntos. El jugador decide dividir porque ambas cartas tienen el mismo valor.

Desarrollo: el jugador separa las cartas en dos manos independientes y realiza una segunda apuesta de ₡100, igual a la apuesta original.

Mano 1: recibe un 5 y forma [8, 5], con un total de 13 puntos. Luego pide una carta adicional y recibe un 7, alcanzando 20 puntos. Decide plantarse.

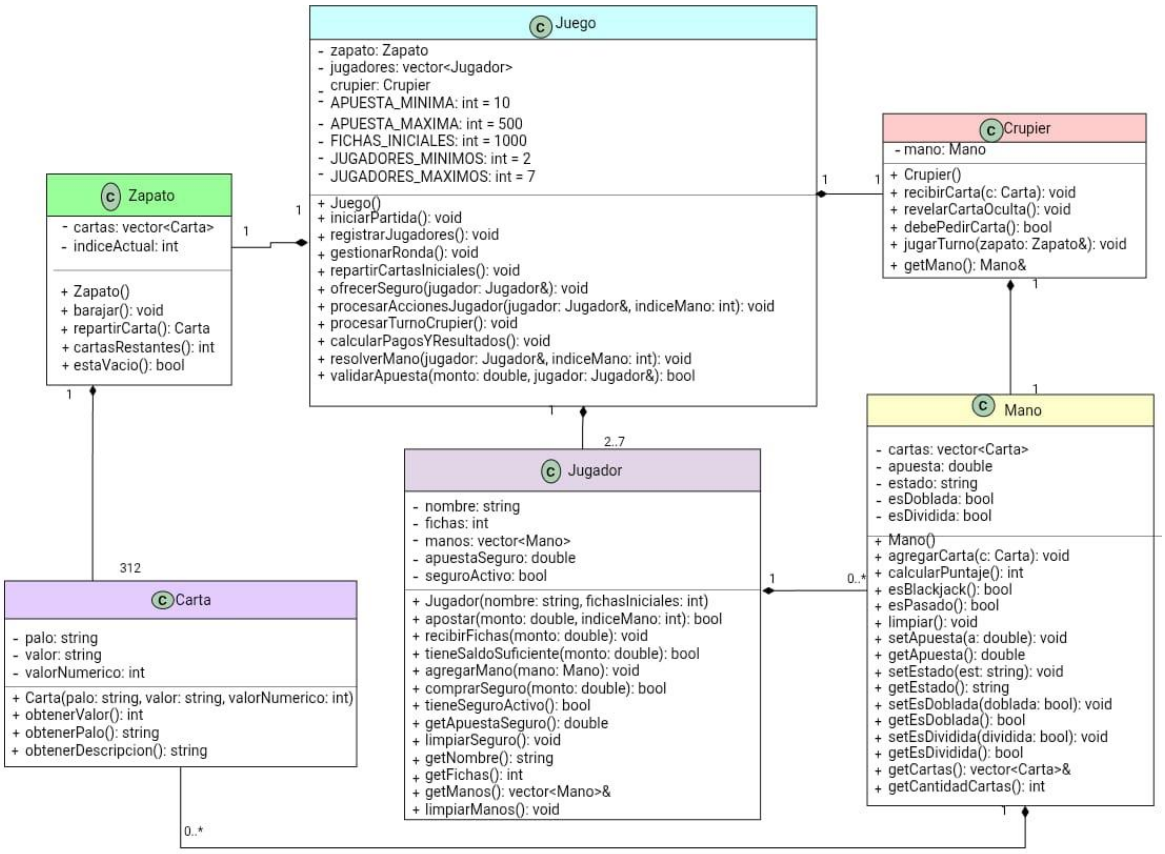
Mano 2: recibe un As y forma [8, As], con un total de 19 puntos. Decide plantarse.

Cartas finales del crupier: total de 18 puntos.

Resultado: las dos manos del jugador superan los 18 puntos del crupier. Ambas resultan ganadoras. El total de 21 puntos, si se obtuviera después de dividir, no se considera Blackjack natural.

Apuesta y pago: el jugador arriesga ₡200 en total, distribuidos en dos apuestas de ₡100. Cada mano gana ₡100 de ganancia neta. La ganancia neta total es de ₡200 y el jugador recibe ₡400 en total, incluyendo las dos apuestas originales.

4. Diagrama de clases UML y descripción de responsabilidades



https://drive.google.com/file/d/12AcnzBD59MfX4_GQ_NGDVLX7d4KBGB0k/view?usp=sharing

Descripción de responsabilidades a alto nivel

- **Carta:** Representa cada naipe de la baraja, encapsulando sus características principales: palo, valor visible y valor numérico. Proporciona métodos para consultar esta información y obtener su descripción.
- **Zapato:** Administra el conjunto de 312 cartas correspondientes a los seis mazos utilizados en la partida. Se encarga de barajar las cartas, controlar el índice de reparto, entregar cartas y verificar cuántas quedan disponibles.
- **Mano:** Administra las cartas correspondientes a un jugador o al crupier, así como la apuesta y el estado de la mano. Se encarga de calcular el puntaje, identificar si existe un Blackjack, determinar si se ha superado el límite de 21 puntos y controlar situaciones especiales como doblar o dividir.
- **Jugador:** Representa al participante de la partida y administra su información personal y financiera, incluyendo su nombre, fichas disponibles y apuestas. Permite realizar apuestas, recibir fichas, comprobar si dispone de saldo suficiente, adquirir seguro y administrar múltiples manos cuando se realiza una división.
- **Crupier:** Representa a la casa y administra su propia mano. Se encarga de recibir cartas, revelar la carta oculta y ejecutar su turno siguiendo las reglas establecidas del Blackjack, determinando cuándo debe solicitar otra carta.
- **Juego:** Funciona como el controlador central (Controller), coordinando el desarrollo de la partida y la interacción entre los demás objetos. Administra el zapato, los jugadores y el crupier; controla el registro de participantes, las apuestas mínimas y máximas, el reparto inicial, el desarrollo de los turnos, la resolución de las manos y el cálculo de los pagos y resultados finales.

5. Plan de acción del proyecto

El desarrollo del proyecto Blackjack se organizó en diferentes actividades, distribuidas en dos fases de trabajo. Estas contemplan el análisis de los requerimientos, el diseño de la estructura del sistema, la programación en C++, las pruebas y la elaboración de la documentación correspondiente.

5.1. Lógica y estructura de Programación Orientada a Objetos (POO)

- Diseño e implementación de las clases base del sistema: “Carta”, “Zapato” y “Mano”.

- Desarrollo de la lógica matemática para el cálculo dinámico del valor de los Ases (1 u 11), así como la administración del zapato compuesto por seis mazos (312 cartas).

- Realización de pruebas de compilación y funcionamiento en consola para verificar la estructura inicial del programa.

5.2. Arquitectura, control y documentación

- Diseño e implementación de las clases encargadas del control y la interacción del juego: “Jugador”, “Crupier” y “Juego”.

- Elaboración, revisión y actualización del diagrama de clases UML, incluyendo atributos, métodos, relaciones y multiplicidades.

- Descripción de las responsabilidades de cada componente del sistema.

- Elaboración y revisión del documento formal en formato PDF, incluyendo introducción, definición del problema, reglas del juego y cinco ejemplos de uso con moneda local (colones costarricenses).

- Configuración del repositorio Git para almacenar el código fuente y facilitar el control de versiones y el trabajo colaborativo.

Cronograma de actividades

El proyecto se desarrolló en dos fases, organizadas de acuerdo con los objetivos y las actividades establecidas por el equipo.

Fase 1: Análisis, diseño UML y avance base

Periodo: del 18 al 30 de octubre de 2026.

Actividad	Descripción
Actividad 1	Definición de las reglas del negocio, análisis de los requerimientos y elaboración de casos de uso prácticos.
Actividad 2	Diseño del diagrama de clases UML y especificación de las relaciones y multiplicidades.
Actividad 3	Implementación de la estructura modular en C++, mediante archivos .h y .cpp, correspondientes a las clases base.
Actividad 4	Elaboración de la documentación inicial, integración de los apartados solicitados y preparación de la exposición.

Resultado de la fase: Se estableció la estructura inicial del proyecto, se desarrolló una parte del código y se elaboraron el documento y el primer diseño UML.

Fase 2: Implementación, integración y ajustes del sistema

Periodo previsto: del 30 de octubre al 6 de noviembre de 2026.

Fecha de entrega actualizada: 13 de noviembre de 2026.

Actividad	Descripción
Actividad 1	Continuación de la implementación de las clases y funcionalidades del Blackjack en C++.
Actividad 2	Revisión y corrección de errores en el código, así como ajustes en la estructura de las clases.
Actividad 3	Actualización del diagrama UML de acuerdo con los cambios realizados en el código, incluyendo los límites de participación de los jugadores.
Actividad 4	Creación y configuración del repositorio Git para organizar el código y llevar el control de versiones.
Actividad 5	Revisión, corrección y actualización de la documentación del proyecto.
Actividad 6	Integración de los avances, revisión general del sistema y preparación de la entrega final.

Resultado de la fase: Se avanzó en la implementación del código, se realizaron correcciones, se actualizó el UML, se configuró el repositorio y se completó la revisión de la documentación.

Nota sobre el cronograma: Inicialmente, se estableció como fecha de entrega el viernes 6 de noviembre de 2026. Posteriormente, se acordó ampliar el plazo hasta el viernes 13 de noviembre de 2026.

Bitácora de trabajo del proyecto

La siguiente bitácora registra las principales actividades realizadas durante el desarrollo del proyecto Blackjack, detallando los periodos de trabajo, las integrantes responsables y los resultados obtenidos en cada etapa.

Registro de actividades.

N.º	Periodo	Actividades realizadas	Responsables
1	18-19 de octubre	Análisis de la rúbrica, investigación de las reglas del Blackjack y definición de los requerimientos del sistema.	Sasha, Daniela y Kris
2	18-30 de octubre	Elaboración del documento inicial, incluyendo introducción, definición del problema, reglas y ejemplos de uso.	Sasha
3	18-30 de octubre	Diseño del primer diagrama de clases UML y organización de las relaciones entre los componentes.	Sasha
4	18-30 de octubre	Desarrollo de una parte del código en C++, mediante la implementación de las clases y sus respectivos archivos “.h” y “.cpp”.	Sasha
5	20-21 de octubre	Revisión del diagrama UML, comprobación de multiplicidades y pruebas de compilación del código base.	Sasha, Daniela y Kris
6	30 de octubre-6 de noviembre	Continuación de la programación, implementación de	Daniela y Kris

		funcionalidades y desarrollo de otras partes del código.	
7	30 de octubre-13 de noviembre	Revisión y corrección de errores, ajustes en las clases y comprobación del código desarrollado.	Daniela y Kris
8	30 de octubre-13 de noviembre	Creación y configuración del repositorio Git para organizar los archivos y facilitar el trabajo colaborativo.	Daniela y Kris
9	30 de octubre-13 de noviembre	Revisión, corrección y actualización del documento del proyecto, incorporando los avances de la segunda fase.	Sasha
10	30 de octubre-13 de noviembre	Reorganización y actualización del diagrama UML, incorporando los cambios en las clases y estableciendo un mínimo de dos y un máximo de tres jugadores por partida.	Sasha
11	11-13 de noviembre	Revisión general del proyecto, organización de las evidencias, verificación de la correspondencia entre el código y el UML y preparación de la entrega final.	Sasha, Daniela y Kris

Nota: Los periodos de la segunda fase corresponden al intervalo de trabajo previsto y su ampliación.

Distribución de responsabilidades

Durante el desarrollo del proyecto, las integrantes distribuyeron las actividades de acuerdo con las necesidades de cada fase, manteniendo la colaboración en las revisiones y los ajustes generales.

Integrante	Responsabilidades y aportes
Sasha (Integrante 1)	Elaboración del documento inicial y sus diferentes apartados, diseño del primer diagrama UML, implementación de una parte del código durante la primera fase y posterior revisión, corrección y actualización del documento y del UML durante la segunda fase.
Daniela (Integrante 2)	Desarrollo de una parte del código en C++, implementación de funcionalidades, revisión y corrección de errores y participación en la creación y configuración del repositorio Git.
Kris (Integrante 3)	Desarrollo de una parte del código en C++, revisión y corrección de las clases, ajustes en la implementación y participación en la creación y configuración del repositorio Git.

Las tres integrantes participaron en el desarrollo del proyecto, contribuyendo desde sus respectivas responsabilidades a la construcción, revisión y organización del sistema Blackjack.

Minutas de trabajo

Minuta n.º 1: Planificación y análisis de requerimientos

Fecha: 18 al 19 de octubre de 2026.

Hora: 2:00 p. m. – 4:15 p. m.

Temas trabajados:

- Análisis detallado de la rúbrica y los requerimientos correspondientes a la primera fase del proyecto.

- Investigación de las reglas del Blackjack, incluyendo el manejo de los seis mazos, los límites de apuestas en colones costarricenses y el comportamiento del As (1 u 11).

- Definición de la estructura modular del programa mediante archivos “.h” y “.cpp”, siguiendo los principios de Programación Orientada a Objetos.

Participantes: Sasha, Daniela y Kris.

Próximo trabajo: Revisión de avances el 21 de octubre de 2026.

Minuta n.º 2: Integración del diseño UML y pruebas de código

Fecha: 20 al 21 de octubre de 2026.

Hora: 1:00 p. m. – 4:00 p. m.

Temas trabajados:

- Revisión conjunta del diagrama de clases UML y validación de las relaciones y multiplicidades entre las clases Juego, Zapato, Jugador y Crupier.

- Realización de pruebas de compilación del código base en C++, verificando el funcionamiento del barajado de los 312 naipes y el reparto inicial.

- Revisión de la documentación y organización de las evidencias del funcionamiento del programa.

Participantes: Sasha, Daniela y Kris.

Próximo trabajo: Continuación del desarrollo y preparación de la exposición el 25 de octubre de 2026.

Minuta n.º 3: Continuación del desarrollo y actualización del proyecto

Periodo: Del 30 de octubre al 13 de noviembre de 2026.

Temas trabajados:

- Continuación de la implementación de las funcionalidades del Blackjack en C++.

- Revisión y corrección de errores en el código desarrollado.

- Creación y configuración del repositorio Git para el almacenamiento y control de versiones del proyecto.

- Actualización del diagrama UML, incorporando las modificaciones realizadas en las clases y estableciendo los límites de jugadores por partida.

- Revisión y corrección de la documentación para mantenerla acorde con el desarrollo actual del sistema.

Participantes: Sasha, Daniela y Kris.

Resultado: Avance de la implementación del sistema, actualización de la estructura UML y organización de los archivos y la documentación para la entrega final.

6. Avance de la implementación

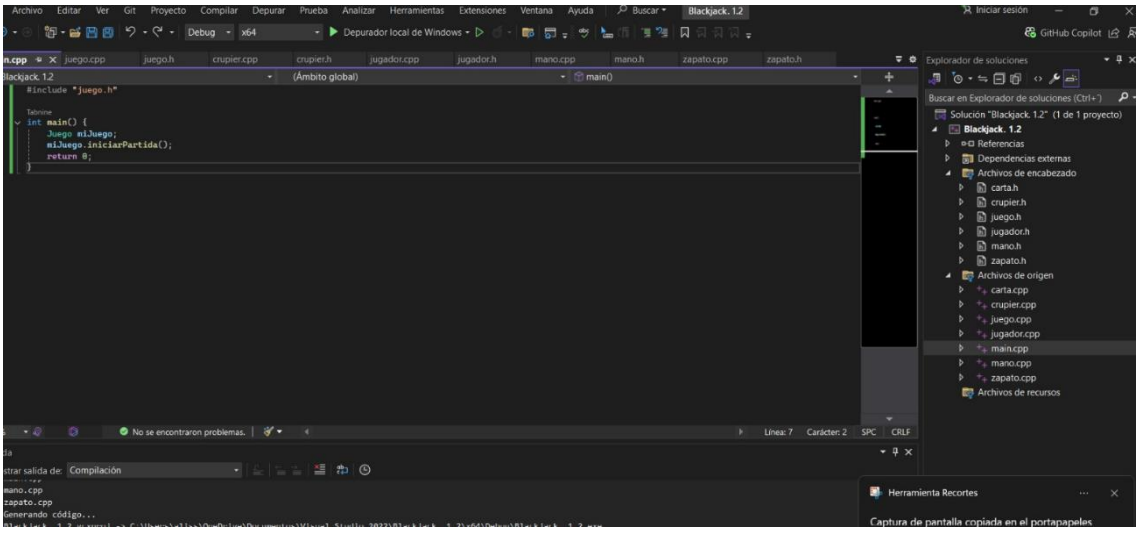


Figura 1. Estructura del proyecto en Visual Studio (Fase 1): Se observa la organización del proyecto en archivos .h y .cpp, correspondientes a las clases del simulador. También se muestra el archivo main.cpp, desde el cual se crea el objeto de la clase Juego y se inicia la partida mediante iniciarPartida().

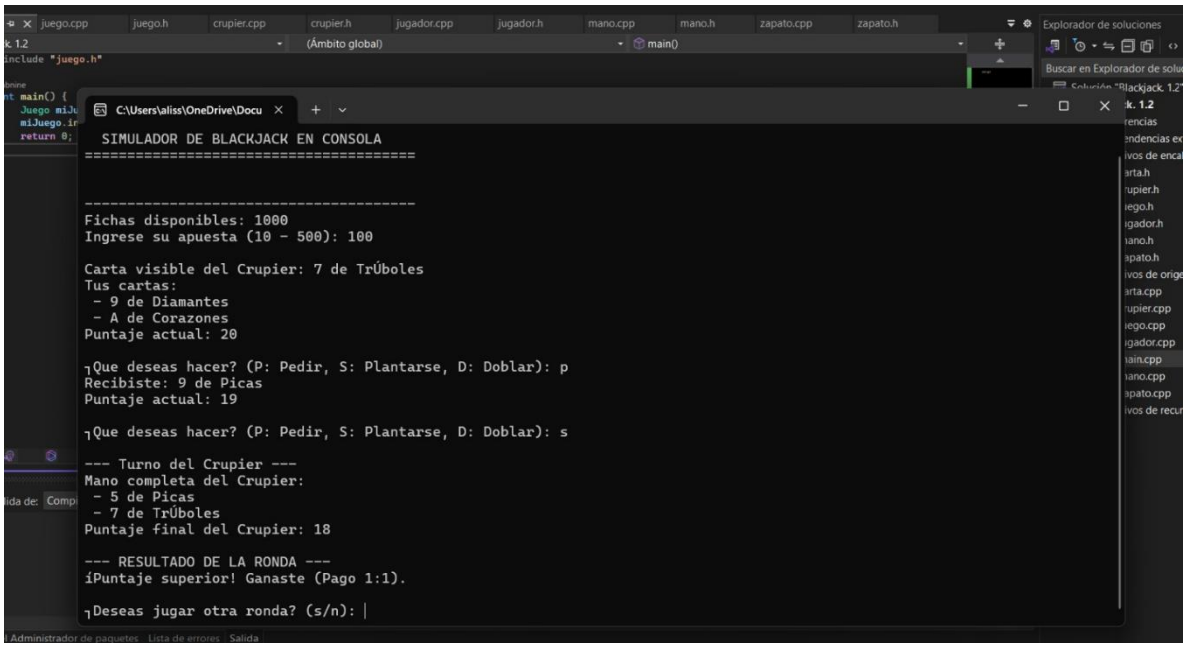


Figura 2. Ejecución del simulador de Blackjack (Fase 1): Se evidencia una partida ejecutada en consola. El jugador realiza una apuesta, recibe sus cartas, solicita una carta adicional y posteriormente decide plantarse. Después se muestra la mano del crupier y el resultado de la ronda, en la que el jugador obtiene la victoria.

```
void Juego::calcularPagosYResultados() {
    Mano& mano = jugador.getManos()[0];
    int ptosJugador = mano.calcularPuntaje();
    int ptosCrupier = crupier.getMano().calcularPuntaje();
    int apuesta = static_cast<int>(mano.getApuesta());
    std::cout << "\n--- RESULTADO DE LA RONDA ---\n";

    if (mano.esBlackjack()) {
        if (crupier.getMano().esBlackjack()) {
            std::cout << "Empate de Blackjack (Push). Recuperas tu apuesta.\n";
            jugador.recibirFichas(apuesta);
        }
        else {
            int ganancia = apuesta + (apuesta * 3 / 2);
            std::cout << "¡BLACKJACK NATURAL! ¡Felicidades! Ganaste " << (apuesta * 3 / 2) << " fichas (Pago 3:
            jugador.recibirFichas(ganancia);
        }
```

Figura 3. Implementación de la lógica para calcular los resultados (Fase 2): Se muestra una parte del método calcularPagosYResultados(), perteneciente a la clase Juego. En este fragmento se evalúan situaciones como el empate entre dos Blackjack y el Blackjack natural, además de calcular las ganancias y actualizar las fichas del jugador.